LTOビルドされたRustバイナリのライブラリ関数識別手法

田中 恭之[1,a)]　松田 亘[1]　中津留 毅[1]

概要：Rust 製実行ファイルは，モノモーフィゼーションやトレイトの静的ディスパッチにより型ごとに関数が実体化されるため，リバースエンジニアリング時の解析対象関数数が著しく増大する．さらに，LTO（Link Time Optimization）を有効にすると，クレート境界を越えた最適化やインライン展開により，ライブラリ関数とユーザ関数の境界が失われ，従来の関数シグネチャやシンボル情報に依存した識別手法の適用が困難となる．本論文では，Windows 上で LTO が有効な Rust バイナリを対象とし，Rust デコンパイルコードの特徴のみを利用してライブラリ関数を識別する手法を提案する．提案手法は，ライブラリ候補の抽出，近傍関数の補完，コールグラフに基づく安全な伝播，意味的まとまりを考慮した再分類の 4 段階で識別範囲を拡張する．評価の結果，従来方式では平均 85%程度であった判定不能ライブラリ関数の割合を平均 25%程度に低減し，ユーザ関数の誤判定率を平均 7%程度に抑制した．提案手法はシンボル名や PDB，既存の他の方式に依存せず，LLVM メジャーバージョンの違いに対しても耐性を有する．本手法により解析対象関数を大幅に削減でき，Rust バイナリ解析の効率化に寄与する．

キーワード：Rust，LTO，リバースエンジニアリング，ライブラリ識別

Library Function Identification for LTO-Built Rust Binaries

Yasuyuki Tanaka[1,a)]　Wataru Matsuda[1]　Takeshi Nakatsuru[1]

Abstract: Rust executables contain a large number of functions due to monomorphization and static trait dispatch. With Link Time Optimization (LTO), cross-crate optimization and inlining further obscure the boundaries between library and user-defined functions, limiting the effectiveness of conventional signature-based identification methods. This paper proposes a method for identifying library functions in LTO-enabled Rust binaries on Windows using only features extracted from decompiled code. The method consists of four stages: library candidate extraction, neighboring function completion, call graph-based propagation, and semantic reclassification. Experimental results reduce the proportion of unidentified library functions from about 85% to 25% while keeping the average false positive rate for user-defined functions below 7%. The proposed method requires neither symbol names nor PDB files and is robust across major LLVM versions, significantly reducing the manual reverse engineering effort for Rust binaries.

Keywords: Rust, LTO, reverse engineering, library identification

1. はじめに

リバースエンジニアリングは，セキュリティ分析やソフトウェア保守など，ソースコードを利用できない実行ファイルの内部構造や動作を把握するための重要な技術である．近年，高速性とメモリ安全性を両立するプログラミング言語として Rust の採用が拡大しており，解析対象となる Rust 製実行ファイルも増加している．

Rust では，同一の処理であっても使用される型ごとに個別の関数が生成されるため，解析対象となる関数数が大幅に増加する．この挙動は，モノモーフィゼーションやトレイトの静的ディスパッチによるものである．さらに，LTO（Link Time Optimization）を有効にしたバイナリでは，クレート境界を越えた最適化やインライン展開が行われるこ

[1] NTT 社会情報研究所
NTT Social Informatics Laboratories
[a)] yasuyukina.tanaka@ntt.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

とで，ライブラリ関数とユーザ関数の境界が曖昧になる．その結果，シンボル情報や関数シグネチャに依存する従来のライブラリ関数識別手法では十分な判定性能を得ることが困難となる．

本研究では，LTO が有効な Rust バイナリを対象として，デコンパイルコードの特徴からライブラリ関数を識別する手法を提案する．提案手法では，ライブラリ候補の抽出に加え，関数間の関係を利用して識別結果を段階的に補完・伝播することで，従来方式では識別が困難であったライブラリ関数の判定性能を向上させる．ライブラリ関数として識別された関数を解析対象から除外できるため，解析者はユーザ実装部分の解析に注力でき，リバースエンジニアリングの効率化に寄与する．

本論文における主要な貢献は以下である．

- シンボル情報や PDB に依存せず，LTO が有効な Rust バイナリを対象としたライブラリ関数識別手法を提案した．
- 従来方式では平均 85%程度であった判定不能ライブラリ関数の割合を平均 25%程度まで削減した．
- ユーザ関数の誤判定率を平均 7%程度に抑えるとともに，誤判定事例を詳細に分析し，今後の改善に向けた課題を整理した．
- rustc/LLVM メジャーバージョンの違いに対しても高い適用性を有することを示した．

2. 関連研究

RUST crate 識別． RIFT[1] は，Rust バイナリからコンパイラや依存 crate 情報を抽出し，同一環境でライブラリを再構築して生成した FLIRT シグネチャにより標準ライブラリや依存 crate の関数を識別し，解析対象を攻撃者実装部分へ絞り込む．一方，LTO ビルドでは関数のインライン化・統合により識別精度が大きく低下する課題がある．Muntean ら [2] は，逆アセンブルした Rust バイナリから Opcode n-gram を抽出し，TF-IDF と Voting により含まれる crate を識別する静的解析手法を提案した．約 150 種類の crate を対象に評価しているが，LTO ビルドへの対応や評価は行われていない．

Rust バイナリ解析支援． Oxidizer[3] は，Rust バイナリを高忠実度な Rust 風疑似コードへ復元する専用デコンパイラを提案した．Rust 特有の型情報，所有権を考慮したデータ構造，列挙型やパターンマッチなどの言語機能を解析・復元することで，従来の C ベースのデコンパイラでは失われる Rust の意味情報を保持し，高い可読性と解析精度を実現している．ReOxide[4] は，Ghidra のデコンパイラを拡張し，Rust 特有のコンパイルパターンを認識することで，Rust バイナリの可読性と解析性を向上させる．GhidRust[5] は，Rust バイナリ解析向けの Ghidra プラグインであり，Function ID による標準ライブラリ関数の識別や Rust 風のデコンパイル表示を提供する．

これらの研究は Rust バイナリの解析支援に焦点を当てているが，LTO 環境下でライブラリ（crate）を識別することを目的とはしておらず，本研究とは目的が異なる．

ライブラリ識別全般． LibDB[6] は，関数埋め込みと関数呼び出しグラフを用いて，バイナリ中のサードパーティライブラリを識別する手法を提案した．FOSSIL[7] は，命令列や制御フローなどの特徴を用いて，バイナリ中のオープンソースライブラリ関数を識別する手法を提案した．BinShape[8] は，関数の CFG や命令列から抽出した特徴量を用いて，ライブラリ関数を高精度に識別する手法を提案した．

3. 実験環境と観察サンプル

3.1 実験環境とビルドオプション

実験環境を以下に示す．4 章で示す提案方式は IDAPython で実装した．

- Windows11 Pro
- rustc 1.94.1(x86-64-pc-windows-msvc)
- cargo 1.94.1
- IDA Pro 9.3 + IDAPython

ビルトオプションは以下とした．

- lto = fat
- codegen-units = 1
- opt-level = 3
- strip = true

上記 lto，codegen-units，opt-level によりコンパイラ最適化とインライン化を最大限適用し，strip 条件によりシンボルおよびデバッグ情報を削除した．これにより，リバースエンジニアリング時に解析が困難となる条件を設定した．なお，前述のとおり Rust ではモノモーフィゼーションおよび静的ディスパッチによりライブラリ関数が多数実体化されるため，識別対象関数数が増加する．

3.2 観察サンプル

Rust/LLVM のデコンパイルコード特徴を観測するサンプルとして合計 10 個用意した．以下の頭の 8 個は自作プログラム，最後の 2 個は github から入手した．このコードを上記環境でビルド後，IDA Pro 9.3 でデコンパイルを行い，関数の特徴を観察し，後述する提案方式を組み立てた．観察に際しては，PDB 情報，RIFT，Lumina[9] も補助的に利用し，可能な限りライブラリ関数名を確認した．ただし，LTO バイナリではこれらの情報のみでは十分な識別が困難なため，デコンパイルコードの特徴を中心に観察を行った．

10 個のコードは以下の特徴を持つ．

std_only_cli：formatting，Vec，String，PathBuf を含む，素の Rust release/LTO に近い基準サンプル．

© 2026 Information Processing Society of Japan

collections_iterators：HashMap, BTreeMap, iterator, closure を含み，alloc / core 由来処理との混ざり方を観測．
generics_traits：generic，trait bound，trait object，enum dispatch を含み，monomorphization と trait dispatch を観測．
filesystem_process：std::fs，std::process::Command，path/env を含み，std の Windows 実装由来 helper の観測．
windows_api_ffi：Win32 API 直叩きで，FFI / Win32 wrapper の典型を観測
serde_json_sample：derive と JSON 処理を含み，proc-macro generated code を観測．
rayon_crossbeam：parallel iterator, channel, thread, sync を含み，thread/sync と crate 固有 heavy helper を観測．
sysinfo_sample：OS 情報列挙系で，WindowsAPI が多用れる crate helper 群の代表例として観測．
server 及び client：github から，リバースエンジニアリング対象として想定される研究目的の RAT(RemoteAccessTool) を選定した．

4. 提案方式

4.1 概要

LTO バイナリに対し，ライブラリらしさを関数本体だけで強く示す関数をヒューリスティックに特定し，seed にする（ダイレクトアンカー）．次に，seed 近傍の微小なヘルパー関数をローカルアンカーとして，seed を拡張する．その後，ダイレクトアンカーとローカルアンカーをコンポーネント seed として，距離・ファミリー条件・ユーザ関数保護，成長制限を設け，安全にプロパゲーションし，同一ファミリーのライブラリ関数群を段階的に回収する．広いファミリー分類で，できるだけ多くライブラリ関数を回収し，その後，狭いファミリー分類で意味の強い単位に再分類する．関数本体から得られる証拠と，周辺関数との接続関係を段階的に組み合わせることにより，強い証拠を持つ関数を起点にしつつ，単独では証拠が弱い小関数も，近傍関係を用いて安全に回収できる．最終的に決定するファミリー一覧を表 1 および表 2 に示す．尚，ファミリーはバイナリ上の特徴に基づく機能的分類であり，crate 境界やコード由来を直接示すものではない．LTO 等の最適化により境界が融合するため，ファミリー名は観測されたコード形状に基づく推定である．この点は 5 章で評価する．

4.2 ダイレクトアンカー特定

ダイレクトアンカー特定では，関数単体から取得可能な特徴を用いて，ライブラリ関数の候補を抽出する．まず各関数について，imports, 文字列, デコンパイルコード, 関数サイズ，basic block 数，fan-in / fan-out，caller / callee などの特徴を取得する．これらの特徴のうち，特に Imports に含まれる API の特徴からライブラリファミリーの候補を抽出する．ただし，特定の API を利用しているという理由だけでは，ライブラリ関数とは判定しない．例えば，メモリ確保 API を呼び出す関数であっても，関数本体が大きく複数の処理を含む場合には，ユーザ実装の一部である可

表 1 broad family 一覧

ファミリー名	説明
alloc_cleanup	GetProcessHeap/HeapAlloc/HeapFree/HeapReAlloc を手掛かりに，メモリ確保・解放・再確保の薄い補助関数群.
fmt_output	WriteConsoleW，GetStdHandle，文字コード変換を手掛かりに，コンソール出力や文字列整形の補助関数群.
cleanup_drop	InterlockedDecrement，HeapFree の反復，drop 風の後始末処理を手掛かりに，資源解放側の補助関数群.
sync_lifetime	InterlockedDecrement や WaitOnAddress/WakeByAddress*を手掛かりに，参照寿命管理や wait/wake 系の小関数.
rustc_demangle	invalid syntax，punycode，backref 等の文字列と深い call graph 位置を手掛かりに，Rust demangle/parser 系.
fmt_core	fmt::error，formatter，間接 writer 操作，数字テーブルを手掛かりに，importless な formatter 中核処理.
async_atomic_waker	atomic 操作と waker vtable 参照を手掛かりに外部 API を呼ばない async waker 補助処理.
async_timer	timer already fired，timers are disabled，時刻計算定数などを手掛かりに，async timer の状態処理.
wait_wake_runtime	WaitOnAddress/WakeByAddress*と atomic compare/exchange を手掛かりに，wait/wake runtime 処理.
crossbeam_select_wait	Crossbeam channel の select において、待機期限の管理、待機状態の原子的な確定、通知後の登録解除を行う内部ヘルパー群.
tokio_channel_queue	HeapFree の反復，InterlockedDecrement64，queue block 参照を手掛かりに，Tokio channel queue cleanup.
process_introspection	GetAdaptersAddresses，DeviceIoControl 系 API などを手掛かりに，process/system 情報取得処理.
runtime_support	thunk，guard check，backtrace，環境変数取得，例外処理 API を手掛かりに，CRT/runtime 支援処理.

© 2026 Information Processing Society of Japan

表 2　narrow family 一覧

ファミリー名	説明
refcount_release	atomic decrement と HeapFree を手掛かりに，参照カウントを減らして最後に解放する処理
fmt_output	task state 文字列と InterlockedCompareExchange64/ExchangeAdd64 を手掛かりに, async task raw 状態処理
async_waker	waker 関連文字列，refcount step，vtable 操作を手掛かりに，async waker の参照管理処理

能性がある．そのため，候補の抽出には，API の利用状況に加えて，関数サイズや basic block 数，fan-in / fan-out などの構造的特徴を用いる．各ライブラリファミリーについて，これらの特徴に対する判定条件を個別に設定し，条件を満たす関数のみを seed として採用する．このように，ダイレクトアンカーでは，API 特徴による候補抽出と関数の構造的特徴による絞り込みを組み合わせることで，ライブラリ関数としての役割が比較的明瞭な小規模関数を seed として特定する．なお，ライブラリファミリーごとの詳細な判定条件は，各ファミリーの特徴に応じて個別に設定している．

- GetProcessHeap と HeapAlloc のみを呼び，関数サイズや fan-out が小さい関数は，ユーザ処理本体ではなくメモリ確保処理を包む薄い補助関数とみなし，alloc_cleanup の seed とする．
- WriteConsoleW や GetStdHandle，文字コード変換 API を中心に呼ぶ小型関数は，コンソール出力や整形出力の補助関数とみなし，fmt_output の seed とする．
- 外部 API を呼ばなくても，formatter の間接 writer 操作や数字テーブル参照のような特徴を持つ小型関数は，importless な整形処理の中核とみなし，fmt_core の seed とする．

ダイレクトアンカー特定は，後述するプロパゲーションの高信頼 seed を確定し，seed 汚染や user 関数誤吸い込みを抑える目的も担う．

4.3　ローカルアンカー特定

近傍を seed 化する．4.2 で確定したダイレクトアンカーを起点に， caller / callee の 1-hop 近傍関数を探索する．近傍関数を import の少なさ，サイズ，fan-out などファミリー毎の選定条件で絞り込みを実施．絞り込みの結果，小さく単純な helper / local bridge だけが残り，ユーザ関数は除外される．条件を満たす関数をローカルアンカーとして決定し seed に追加する．これによりダイレクトアンカーだけでは拾えない小関数も回収しやすくする．ローカルアンカーは，後段のプロパゲーションで初めて昇格させる関数とは異なり，強い seed の近傍に存在すること自体を追加証拠として使う．例えば，外部 API を呼ばず，短い本体で，特定の補助処理だけを他の関数へ橋渡ししている関数は，単独では証拠が弱くても，確定済み seed の直近にある場合には同一系統の補助関数として扱える．

Algorithm 1 Guarded Library Component Propagation and Promotion

Require: Feature set $\mathcal{F}$, component sets $\mathcal{C}$, current classification results $\mathcal{R}$
Ensure: Updated classification results $\mathcal{R}$ and component sets $\mathcal{C}$

1: $\mathcal{L} \leftarrow$ ExtractLibraryFamilies($\mathcal{C}$)
2: **for all** $fam \in \mathcal{L}$ **do**
3: $\quad distanceMap[fam] \leftarrow BuildDistanceMap(fam, 2)$
4: **end for**
5: **repeat**
6: $\quad changed \leftarrow false$
7: $\quad$ **for all** $f \in \mathcal{F}$ **do**
8: $\quad\quad$ **if** Skip(f) **then**
9: $\quad\quad\quad$ **continue**
10: $\quad\quad$ **end if**
11: $\quad\quad (fam, score) \leftarrow BestFamily(f)$
12: $\quad\quad$ **if** $fam = None$ **or** GrowthCap(fam) **then**
13: $\quad\quad\quad$ **continue**
14: $\quad\quad$ **end if**
15: $\quad\quad neighbors \leftarrow NeighborCount(f, fam)$
16: $\quad\quad dist \leftarrow distanceMap[fam][f]$
17: $\quad\quad$ **if** $score \geq promotionThreshold$ **and** CanPromote($f, fam, neighbors, dist$) **then**
18: $\quad\quad\quad$ Promote(f, fam)
19: $\quad\quad\quad distanceMap[fam] \leftarrow BuildDistanceMap(fam, 2)$
20: $\quad\quad\quad changed \leftarrow true$
21: $\quad\quad$ **else if** $score \geq weakThreshold$ **and** CanMarkLikely(f, fam) **then**
22: $\quad\quad\quad$ MarkLikely(f, fam)
23: $\quad\quad$ **end if**
24: $\quad$ **end for**
25: **until** $changed = false$
26: **for all** $f \in \mathcal{F}$ **do**
27: $\quad$ **if** ClusterLeafCondition(f) **then**
28: $\quad\quad$ PromoteClusterLeaf(f)
29: $\quad\quad distanceMap[BestFamily(f)] \leftarrow BuildDistanceMap(BestFamily(f), 2)$
30: $\quad$ **end if**
31: **end for**
32: **for all** $f \in \mathcal{F}$ **do**
33: $\quad$ **if** SecondPassCondition(f) **then**
34: $\quad\quad$ PromoteLikely(f)
35: $\quad\quad distanceMap[BestFamily(f)] \leftarrow BuildDistanceMap(BestFamily(f), 2)$
36: $\quad$ **end if**
37: **end for**

4.4　ガード付きプロパゲーションと昇格

プロパゲーションの流れを Algorithm 1 に示す．4.2,4.3 の seed から開始し (L1-3)，昇格した member を逐次探索起点へ追加しながら (BuildDistanceMap, L3, 19, L29, L35)，同一 library family に属すると考えられる補助関数を段階

© 2026 Information Processing Society of Japan

的にクラスタへ追加する (L5-L25)．コールグラフ全体へ一度に伝播するのではなく (L15,16)，既存 member 近傍の関数のみを候補とし，複数の証拠を組み合わせて安全にクラスタを成長させる (L17,L18)．

各候補関数については，全 library family に対する親和性スコアを算出し，最もスコアの高いファミリーを選択する (BestFamily, L11)．スコアは，同一ファミリーの既存 member との隣接数，既存 member からの距離，runtime root からの距離，残差文字列の有無，caller が既に library/runtime 側へ閉じているか，およびファミリー固有の shape・import・文字列・デコンパイル特徴などを総合して計算し，競合する特徴は減点要素として扱う．単一の特徴のみでは昇格を行わず，複数の証拠が一致した場合のみ library member へ昇格させる．

一方で，一定長以上の意味ある文字列を保持する関数や，FFI 関連 import を利用する関数，runtime 近傍の大型 orchestrator など，ユーザ固有ロジックである可能性が高い関数は伝播対象から除外する．また，各ファミリーには成長上限（growth cap）を設け，過度なクラスタ拡大を防止する (L8-13)．以下に代表的な例を示す．

- **alloc_cleanup** では，HeapAlloc や HeapFree を直接呼ぶ seed 近傍の小型関数を候補とする．ただし，HeapFree を呼ぶだけでは昇格させず，既存 member との隣接数，seed からの距離，caller の閉包性，残差文字列の有無などを総合的に評価し，メモリ管理補助関数として閉じた構造を形成する場合のみ library へ昇格する．
- **fmt_core** では，外部 API を持たない整形処理が多いため，formatter の間接 writer 操作，数字テーブル参照，format error 文字列などの body 特徴に加え，既存 member との距離や隣接関係，caller が library/runtime 側に閉じているかを評価し，標準ライブラリ内部の整形補助関数を段階的に回収する．
- **wait_wake_runtime** では，WaitOnAddress や WakeByAddress 系 API，atomic compare/exchange などを含む関数を候補とする．ただし，これらはユーザ実装にも現れるため，seed 近傍であること，既存 member との接続性，caller が library/runtime 側に閉じていることなどを併せて要求し，runtime 内部の同期補助関数のみを library へ追加する．

通常のプロパゲーションでは，十分なスコアとファミリー固有の昇格条件を満たした関数のみを library へ昇格させる．一方，ファミリーとの親和性は認められるものの証拠が十分でない関数は，公開ラベルを unknown のまま維持し，内部状態として `likely_library` に保留する．

その後，Cluster Leaf Promotion では，既存クラスタ近傍に位置する leaf/helper 関数を対象として再評価を行い，通常のプロパゲーションでは回収できなかった小規模な補助関数を追加する (L27,28)．最後に，Likely Library Promotion では，`likely_library` として保留された関数のみを対象に 2nd pass を実施し，通常より厳しいファミリー固有の安全条件を満たした場合に限って library へ昇格させる (L33,34)．これにより，命令列の類似だけでなく，コールグラフ上の位置関係や周辺構造も考慮しながら，標準ライブラリ内部の補助関数を高い精度で回収できる．

4.5 broad から narrow family へ再分類

意味の強い単位に細分化する．前節までに得たファミリー（broad family と呼ぶ）でライブラリとして回収できた関数について，narrow 再分類の対象にする．narrow family ごとに専用のダイレクトアンカーを用いて seed を立て，refcount_release，async_task_raw 等の意味の強い群の起点を先に確定する．残りのライブラリ関数について，seed からの距離，近傍 member 数，family caller / callee，関数構造に関する特徴などを使って narrow 親和性を採点．ファミリーごとの閾値と昇格条件に加え，user 保護や成長上限（growth cap）を満たす場合だけ narrow family に再分類し，変化がなくなるまで反復する．これにより，broad で確保したカバレッジを保ちながら，ライブラリ関数をより意味の強い単位へ細分化できる．

ここで broad family は，ライブラリ関数を広く安全に回収するための粗い分類である．一方，narrow family は，既にライブラリと判定された関数群の内部で，参照カウント解放や async_waker のように意味の強い反復パターンを切り出すための細分類である．したがって narrow 再分類は，カバレッジを無理に拡張する処理ではなく，回収済みライブラリ関数の意味付けを精密化する処理として位置付ける．

4.6 降格

一度ライブラリと判定された関数であっても，後処理で，ライブラリ関数としては証拠が弱いと判断された場合，unknown に戻す降格処理を行う．この処理は，プロパゲーションによって取り込まれた関数のうち，明示的な API 呼び出し，関数構造に関する特徴，固有文字列などの強い意味的証拠を持たず，コールグラフ上の近接性や構造的類似性だけでライブラリ関数と見なされた候補を主な対象とする．

上記の候補を対象に，root 近傍に位置し，fan-out が大きく caller が少ない hub 形状を有する関数，または同様の構造を持つクローンが複数存在する関数を抽出する．これらは，ユーザ関数かライブラリ関数かの判定が困難な内部 bridge/helper である可能性が高い．そこで，ライブラリ関数から除外され，公開ラベルは unknown に戻される．この処理は，分類範囲を広げるためではなく，広めのプロパゲーションによる過分類を抑え，ライブラリ関数の推定を

© 2026 Information Processing Society of Japan

安定的にするための，最後に実施されるプロセスである．

4.7 出力

最終的に，ファミリーごとの関数名，関数ごとの概要情報，関数ごとの詳細情報が JSON 形式で出力される．ここでは，関数ごとの概要情報を Listing 1 に示す．概要情報には，関数名・ファミリー名の他に，どの工程で決定したかを示す，decision_stage，具体的な証拠の例を示す，evidence_tags が含まれる．

Listing 1: 関数概要 JSON の例

```
{
  "addr": "0x14000498B",
  "name": "sub_14000498B",
  "label": "library",
  "component_family": "alloc_cleanup",
  "confidence": "medium",
  "decision_stage": "
      library_component_propagation",
  "reason": "library-first propagation
      assigned function to alloc_cleanup
      component",
  "evidence_tags": [
    "alloc_cleanup",
    "component_propagated"
  ],
  "unknown_reason": null
},
```

5. 評価と考察

5.1 評価 1: ベースライン

3.2 で示した，観察サンプル 10 個の評価結果を，提案手法と RIFT について，それぞれ，表 3，表 4 に示す．Funcs は，デコンパイル結果の総関数数，Lib は提案手法によりライブラリ判定された関数数，Unk(Unknown) は提案手法ライブラリともユーザとも判定できなかった関数数，Unk(%) は，Funcs を母数とした Unk の割合（以下，Unknown 率）である．関数冒頭に述べた様に LTO ビルドバイナリでは，RIFT の Unknown 率が平均 85.64%と大きく上がってしまっていることがわかる．一方で，提案方式の Unknown 率は平均 29.57%となった．これは，提案手法は LTO ビルドバイナリについて，約 7 割のライブラリ識別ができており，リバースエンジニアリング作業の効率化が期待できる．なお，ベースライン評価では，観察フェーズとは異なり，PDB 情報，RIFT，Lumina 等の補助情報は使用していない．アプリ内訳を見ると，server および client が，提案方式および RIFT で下がっていることがわかる．これは自作のコードに比べると，多機能で関数数も大きくなっていたためその点が影響している可能性がある．

表 3 ベースラインの結果 (提案方式)

App	Funcs	Lib	Unk	Unk(%)
collections_iterators	415	342	70	16.87
filesystem_process	713	549	161	22.58
generics_traits	377	312	62	16.45
rayon_crossbeam	950	793	154	16.21
serde_json_sample	539	417	119	22.08
std_only_cli	420	343	74	17.62
sysinfo_sample	1024	835	186	18.16
windows_api_ffi	366	305	58	15.85
server	10847	7152	3692	34.04
client	3411	2348	1060	31.08
Total	19062	13396	5636	29.57

表 4 ベースラインの結果 (RIFT)

App	Funcs	Lib	Unk	Unk(%)
collections_iterators	415	91	324	78.07
filesystem_process	713	153	560	78.54
generics_traits	377	89	288	76.39
rayon_crossbeam	950	158	792	83.37
serde_json_sample	539	109	430	79.78
std_only_cli	420	102	318	75.71
sysinfo_sample	1024	152	872	85.16
windows_api_ffi	366	88	278	75.96
server	10847	1343	9504	87.62
client	3411	452	2959	86.75
Total	19062	2737	16325	85.64

5.2 評価 2: ホールドアウト

ホールドアウトとして，新たに 10 個アプリを選定した．選定観点として，ベースライン評価の結果から，規模が大きいと提案方式で困難になる可能性があることから，ある程度の関数規模があるもの，3.1 で示したビルド環境でビルド可能なもの，ソースコードおよびデコンパイルコードの比較により人手によりユーザ関数が一定数特定できたもの，を選定した．また，リバースエンジニアリング活用シーンの一つとしてマルウェア解析があることから，ランサムウェア含む悪性なアプリを 4 つ含んだ (ransomw, VEN0m, note, Andid0te)．すべて github から入手可能である．表 5 に提案手法による評価結果を示す．Func, Lib, Unk, Unk(%) は，表 3，表 4 と同じ定義であり，User は，デコンパイル結果から，人手によりユーザ関数と特定できた関数の個数を，FP は本提案方式で，ユーザ関数を誤ってライブラリ関数と判定したものの関数の個数を，FP(%) は，User を母数とした FP の割合である．Unknown 率はアプリによりばらつきがあり，14.45〜35.60% と変動し，平均すると 21.51%となった．eza で最も高くなった．unknown ファミリーには，既定義ファミリーでは十分に特徴付けられないライブラリ関数に加え，本方式では特定していない

© 2026 Information Processing Society of Japan

アプリケーション固有のユーザ関数が含まれる．したがって，この差はアプリケーション固有コード，依存ライブラリの構成差，および既定義ファミリーのカバレッジを併せて反映する．FP 率も同じくアプリによりばらつきがあり，平均で，6.95%となった．

表 5 ホールドアウトの結果

App	Funcs	Lib	Unk	Unk (%)	User	FP	FP (%)
bore	4007	3346	658	16.42	15	1	6.67
hurlfmt	5562	4525	1034	18.59	6	0	0.00
hurl	12155	10017	2135	17.56	9	0	0.00
eza	6588	4240	2345	35.60	12	1	8.33
delta	11774	8632	3139	26.66	16	3	18.75
bandwhich	7294	6237	1054	14.45	19	3	15.79
ransomw	552	433	116	21.01	3	0	0.00
VEN0m	1180	923	254	21.53	7	0	0.00
note	6345	5172	1170	18.44	5	1	20.00
Antid0te	810	606	201	24.81	3	0	0.00
Total/Avg.	56267	44131	12106	21.51	95	9	6.95

次に FP であった 9 件について分析を行った結果を表 6 に示す．アドレスは該当関数のアドレスで先頭 0x1800 は共通のため省略している．Pred.Fam. は，本手法によって誤判定されたファミリー名である．また Imp. に，この関数がアプリの挙動を知る上での重要度を High，Middle，Low で，その理由を Rationale に示した．また参考までに Size は該当関数のデコンパイルコードのバイト数として掲載した．重要度が High のものが全体 9 件のうちの 5 件あった．リバースエンジニアリング時にこれを見逃してしまうと対象アプリの挙動理解が損なわれる恐れがあるため重大である．例として，アプリ bore の，Protocal analysis は，クライアントメッセージのシリアライズ・送信処理で，通信プロトコルの理解に必要となる．Middle の例として，アプリ eza の，Layout helper は，端末幅を取得して表示レイアウトに影響する処理だが，ファイル一覧や色付けの処理ではなく，中核の処理では無い．アプリ delta の Low は，Rendering helper で，描画 helper であり，UI の細部に影響するが主要な処理フローには影響しない．また表には無いが，9 件のうち 4 件がダイレクトアンカーであり，残り 5 件がプロパゲーションであった．いずれもファミリー別の詳細条件を調整することで誤判定をなくすことが可能であるが，本方式の制約により，Unknown 率とのトレードオフとなり，今回は Unknown 率が現在の 21.51%からどう変化するかどうかは算出できていない．

図 1 にホールドアウトアプリ毎の，ファミリー種別の割合を示す．alloc_cleanup および runtime_support が 57.54–73.57% を占め，これらが全アプリで主要なファミリーとなった．一方，それ以外のファミリー構成はアプリごとに異なり，cleanup_drop，refcount_release，および fmt_core が比較的高い割合を示した．個別アプリに着目すると，ransomw における fmt_core の比率が高い要因として，println!，eprintln!，format!，writeln! の多用が考えられる．これらは暗号化，ファイル探索，エラー処理，および進捗表示で用いられており，std::fmt の writer/formatter 系ヘルパが生成・残存しやすい構成となっているためと考えられる．また，refcount_release ファミリーについて，bandwhich，note，bore アプリ等で割合が多いが，ソースから Tokio 等の非同期・共有所有権・ランタイム依存を含む構成との差として考えられる．

表 6 FP となった 9 件の関数分析

App	Addr.	Pred. Fam.	Imp.	Rationale	Size
bore	42800	cleanup_drop	H	Protocol analysis	810
eza	565C0	fmt_output	M	Layout helper	123
delta	70400	cleanup_drop	M	Asset loading	1638
delta	57BF0	cleanup_drop	H	Core processing	1571
delta	6F3D0	alloc_cleanup	L	Rendering helper	845
bandwhich	3B0F0	async_timer	H	Result presentation	7496
bandwhich	39C20	async_timer	H	Raw event handling	3365
bandwhich	3DC30	async_timer	H	TUI event handling	3365
note	037D0	alloc_cleanup	M	State initialization	95

5.3 評価 3:LLVM 変化耐性

Rust コンパイラは LLVM をバックエンドとして利用しており，LLVM バージョンの変化に伴う最適化処理やコード生成の差異により，生成されるバイナリおよびデコンパイル結果が変化し，提案方式の性能に影響を与える可能性がある．そこで，1 アプリ（collections_iterators）のみではあるが，異なる LLVM メジャーバージョンに対応する rustc バージョンを用いて，同一のコンパイルオプションでバイナリを生成し，提案方式を評価した．表 7 に結果を示す．LLVM バージョンの更新に伴い，総関数数 (Funcs) は 490 から 395 まで変化しており，コード生成結果が変化していることが確認できる．一方で，unknown 率は 16.46〜27.56%の範囲で推移し，大幅な性能低下は確認されなかった．このことから，提案方式は LLVM のメジャーバージョン更新によるコード生成の差異に対して一定の頑健性を有することが示唆される．

表 7 rustc/LLVM バージョン変化の結果

rustc	Release	LLVM	Funcs	Lib	Unk	Unk (%)
1.69.0	2023-04	15	490	381	106	21.63
1.70.0	2023-06	16	485	378	104	21.44
1.73.0	2023-10	17	468	336	129	27.56
1.81.0	2024-09	18	420	323	94	22.38
1.84.1	2025-01	19	414	335	76	18.36
1.87.0	2025-03	20	424	344	77	18.16
1.94.1	2026-03	21.1	415	342	70	16.87
1.96.0	2026-05	22.1	395	327	65	16.46

© 2026 Information Processing Society of Japan

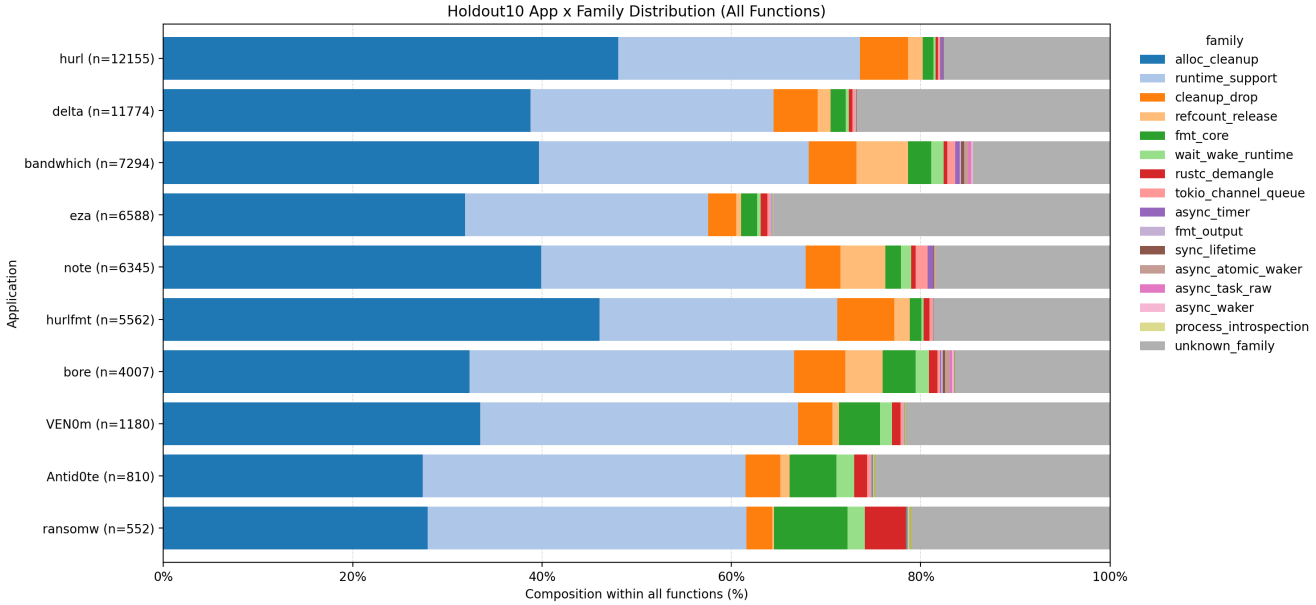


図 1　ファミリー種別の割合

6. 議論

問題設定の限界．提案方式でのファミリーは，バイナリ上の特徴に基づく機能的分類であり，crate 境界やコード由来を直接示すものではない．LTO 等の最適化により境界が融合するため，ファミリー名は観測されたコード形状に基づく推定である．冒頭に述べた Rust 及び LTO の特性から，コンパイルされたバイナリのユーザ関数をライブラリ関数を低レイヤ（デコンパイル/ディスアセンブリレベル）で完全に分割することは困難なケースが存在することは考慮する必要がある．

実環境への適用検証．リバースエンジニアリングが必要なケースとしてマルウェア解析があるが，著者らが日常的に実施しているマルウェア解析においても，Rust 製マルウェアに遭遇する機会が増加しており，既存方式 (RIFT) では十分な効果が得られない LTO が有効と推測される検体も確認している．今回はソースコードがあるホールドアウト検体を用いたが，本来はこのようなソースコードのない実環境でのサンプルを対象にした評価も含めるべきである．そのバイナリに対する定量評価には，正確な Ground Truth を備えたデータセットの構築が必要となる．しかし，その作成には専門的な手作業を要するため容易ではない．本論文では評価対象として含めなかったが，実運用での重要な課題であり，今後取り組む予定である．

7. おわりに

本稿では，LTO が有効な Rust バイナリを対象に，シンボル情報や PDB に依存せず，デコンパイルコードの特徴からライブラリ関数を識別する手法を提案した．評価の結果，判定不能ライブラリ関数の割合を従来方式による平均 85%程度から 25%(ベースラインとホールドアウトの平均値）程度まで低減し，ユーザ関数の誤判定率を平均 7%程度に抑制した．また，rustc/LLVM のメジャーバージョンの違いに対しても高い耐性を示した．今後は実環境における LTO ビルトマルウェア検体に取り組む予定である．

参考文献

[1] https://www.microsoft.com/en-us/security/blog/2025/06/27/unveiling-rift-enhancing-rust-malware-analysis-through-pattern-matching/

[2] Denisa Muntean and Ciprian Oprisa: Software Composition Analysis on Rust Binaries, Proceedings of the 21st IEEE International Conference on Intelligent Computer Communication and Processing (ICCP), 2025, pp. 1–8.

[3] Yibo Liu, Zion Leonahenahe Basque, Arvind S. Raj, Chavin Udomwongsa, Chang Zhu, Jie Hu, Changyu Zhao, Fangzhou Dong, Adam Doupé, Tiffany Bao, Yan Shoshitaishvili, and Ruoyu Wang: Oxidizer: Toward Concise and High-fidelity Rust Decompilation, 2026 IEEE Symposium on Security and Privacy (SP), 2026.

[4] https://reoxide.eu/

[5] https://git.pvv.ntnu.no/oysteikt/GhidRust

[6] Wei Tang, Yanlin Wang, Hongyu Zhang, Shi Han, Ping Luo, and Dongmei Zhang: LibDB: An Effective and Efficient Framework for Detecting Third-Party Libraries in Binaries, Proceedings of the 19th International Conference on Mining Software Repositories (MSR), 2022.

[7] Yaniv David, Nimrod Partush, and Eran Yahav: FOSSIL: A Resilient and Efficient System for Identifying FOSS Functions in Malware Binaries, ACM Transactions on Privacy and Security, Vol. 21, No. 2, 2018.

[8] Jiang Ming, Dinghao Wu, and Peng Liu: BinShape: Scalable and Robust Binary Library Function Identification Using Function Shape, Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA), 2017.

[9] https://docs.hex-rays.com/ida-9.3/add-ons/lumina

© 2026 Information Processing Society of Japan